PART A

1. What is the key objective of project management?

    - B) Delivering the project within cost, time, and performance constraints

2. Which of the following is an essential metric in software project planning?
    - A) Daily code commits
    - B) Function points
    - C) Team size
    - D) Programming language choice

3. Which testing technique begins at the component level and moves outward?
    - A) System testing
    - B) Integration testing
    - C) Unit testing
    - D) Smoke testing

4. Validation testing primarily focuses on:
    - A) Testing internal logic
    - B) Ensuring that the product meets customer requirements
    - C) Debugging code
    - D) Optimizing software performance

5. Cyclomatic complexity is used to:

    - B) Determine the number of independent paths in a program

6. Which of the following is an example of black-box testing?
    - A) Testing internal data structures
    - B) Boundary value analysis
    - C) Basis path testing
    - D) Control structure testing

7. What is a key purpose of software configuration management?
    - A) To improve runtime efficiency
    - B) To maintain the integrity of software artifacts
    - C) To debug all errors in a program
    - D) To train new team members

8. Which tool is commonly used for version control in software projects?
    - A) JIRA
    - B) Git

- C) Jenkins
- D) Selenium

9. Which of the following best describes risk management in software projects?
   - A) Avoiding all potential issues during development
   - B) Identifying, analyzing, and mitigating potential project risks
   - C) Testing software for vulnerabilities
   - D) Ensuring 100% bug-free software
10. In configuration management, which activity ensures the traceability of changes?
    - A) Versioning
    - B) Code refactoring
    - C) Debugging
    - D) Testing

Part B : Short Answer Questions

1. Explain the primary objectives of project management.

   The primary objectives of project management are to deliver a project within the defined scope, time, and budget while meeting quality standards. It involves planning, organizing, executing, and controlling resources to achieve specific goals.
2. Discuss the "Four P's" in project management and their significance.
   - People: The team and stakeholders involved in the project.
   - Process: The methodologies and workflows used to manage the project.
   - Product: The deliverables and outcomes of the project.
   - Project: The overall management of tasks, timelines, and resources.
3. What are common obstacles in managing software projects, and how can they be addressed?

   Common obstacles include scope creep, unrealistic deadlines, poor communication, and resource constraints. These can be addressed through clear requirements, regular communication, risk management, and agile methodologies.
4. Compare different estimation techniques for project planning.
   - Expert Judgment: Relies on the experience of experts.
   - Analogous Estimation: Uses data from similar past projects.
   - Parametric Estimation: Uses statistical models and historical data.
   - Bottom-Up Estimation: Breaks down tasks into smaller components for detailed estimation.

5. Define "function points" and explain how they are used in software estimation.

   Function points measure the functionality provided by software based on user requirements. They are used to estimate project size, effort, and cost by quantifying the software's features.

6. What are the main principles of measurement in software project management?
   - Relevance: Metrics should align with project goals.
   - Accuracy: Data should be precise and reliable.
   - Consistency: Metrics should be applied uniformly.
   - Actionability: Metrics should provide insights for decision-making.

7. How does the critical path method help in project scheduling?

   The critical path method identifies the longest sequence of dependent tasks, determining the minimum project duration. It helps prioritize tasks and manage delays.

8. Differentiate between verification and validation in software testing.
   - Verification: Ensures the software is built correctly (e.g., code reviews, unit testing).
   - Validation: Ensures the correct software is built (e.g., user acceptance testing).

9. Explain the advantages of regression testing in maintaining software quality.

   Regression testing ensures that new changes do not introduce defects into existing functionality. It maintains software stability and reliability over time.

10. How does smoke testing contribute to the software development lifecycle?

    Smoke testing verifies that the most critical functionalities work after a build. It helps identify major issues early, saving time and effort.

11. Describe the steps involved in critical path scheduling.
    - Identify all tasks and their dependencies.
    - Estimate the duration of each task.
    - Determine the critical path (longest path of dependent tasks).
    - Monitor and adjust the schedule as needed.

12. Explain "regression testing" and why it is crucial in software projects.

    Regression testing ensures that new code changes do not break existing functionality. It is crucial for maintaining software quality and preventing unintended side effects.

13. What are the key differences between alpha testing and beta testing?

- Alpha Testing: Conducted internally by the development team in a controlled environment.
- Beta Testing: Conducted by end-users in a real-world environment to gather feedback.

14. Discuss the benefits of smoke testing in software development.

    Smoke testing quickly identifies major issues, ensuring that the build is stable enough for further testing. It saves time and resources by catching critical failures early.

15. Explain the difference between white-box testing and black-box testing.
    - White-Box Testing: Focuses on internal structures and code logic (e.g., unit testing).
    - Black-Box Testing: Focuses on external behavior and functionality (e.g., user acceptance testing).
16. How can cyclomatic complexity be used to design test cases?

    Cyclomatic complexity measures the number of independent paths in a program. It helps identify the minimum number of test cases needed to cover all paths.

17. Describe the importance of boundary value analysis in black-box testing.

    Boundary value analysis tests input values at the edges of valid ranges. It is important because many errors occur at these boundaries.

18. What is the role of debugging in the software testing process?

    Debugging identifies and fixes defects in the code. It ensures that the software functions as intended and meets quality standards.

19. Explain the importance of configuration management in software projects.

    Configuration management ensures that all software artifacts are tracked, controlled, and maintained. It prevents inconsistencies and ensures traceability.

20. How does version control help in managing software changes?

    Version control tracks changes to code, enabling collaboration, rollback to previous versions, and management of concurrent modifications.

21. Discuss the role of metrics in software quality control.

    Metrics provide quantitative data to assess quality, track progress, and identify areas for improvement. They support decision-making and ensure alignment with quality standards.

22. Explain the process of managing risks in software engineering.

    Risk management involves identifying, analyzing, prioritizing, and mitigating risks. It includes monitoring risks throughout the project lifecycle to minimize their impact.

23. What is cyclomatic complexity, and how is it calculated?

    Cyclomatic complexity measures the number of independent paths in a program. It is calculated using the formula:

    M=E−N+2P, where E = edges, N = nodes, and P = connected components.

24. How does boundary value analysis improve the effectiveness of black-box testing?

    Boundary value analysis focuses on edge cases, where errors are most likely to occur. It improves test coverage and identifies defects that might be missed otherwise.

25. What are the main debugging techniques, and when should each be used?

    - Print Debugging: Adding print statements to trace execution (simple issues).
    - Interactive Debugging: Using tools to step through code (complex issues).
    - Log Analysis: Reviewing logs to identify errors (runtime issues).

26. What is configuration management, and why is it important in software engineering?

    Configuration management tracks and controls changes to software artifacts. It ensures consistency, traceability, and reproducibility in software development.

27. Describe the purpose of version control tools and provide examples of widely used tools.

    Version control tools manage changes to code, enabling collaboration and tracking history. Examples include Git, SVN, and Mercurial.

28. How can software metrics be used to improve software quality?

    Metrics provide insights into code quality, performance, and defects. They help identify areas for improvement and ensure adherence to quality standards.

29. Explain the process of risk management in software engineering.

    Risk management involves identifying, analyzing, prioritizing, and mitigating risks. It includes monitoring risks throughout the project lifecycle to minimize their impact.

30. What are the key principles of Agile project management, and how do they benefit software development?

    Key principles include iterative development, customer collaboration, and adaptability. Agile benefits software development by delivering value incrementally, responding to change, and improving team collaboration.